

UNIVERSITÉ TÉLUQ

TRAVAIL NOTÉ 3

PRÉSENTÉ À

FATIMA BENSALMA

COMME EXIGENCE PARTIELLE

DU COURS

PROJET EN SCIENCE DES DONNÉES SCI 1402

PAR

GABRIEL FLEURENT-THIFFAULT

22307023

TRAVAIL NOTÉ 3

10 JUIN 2025

Table des matières

Partager avec Github	3
Normes et spécifications	4
GitFlow	4
Commit préfixé	4
Build.....	4
Docs	4
Feat.....	4
Fichier ReadMe.....	4
Structure de dossiers	5
License	5
Partage du répertoire	5
Bibliographie.....	6

Partager avec Github

La modélisation de la solution a été partagée, certes, mais elle a subi des modifications importantes depuis le dépôt du deuxième travail.

Effectivement, des commentaires supplémentaires ont été ajoutés au code, afin de donner davantage d'explication et d'éclaircissement sur des concepts jugés abstraits.

Toutes les importations ont été fusionnées dans une seule cellule, pour faciliter la lecture du code.

De plus, le code a été restructuré selon les 3 étapes principales d'un projet scientifique, ce qui a causé quelques ennuis :

1. Les diagrammes à moustache ont dû être revus, pour que les deux puissent être affichés. En, effet avec la fusion seul le dernier s'affichait. Pour pallier la situation, une solution trouvée sur Stack Overflow a été utilisée.

Celle-ci consistait à utiliser la fonction **subplots** et de mettre en paramètre le nombre de rangées et de colonnes et la forme voulue (Stack Overflow, 2020). Les infobulles de Google Colab ont été utiles pour aider! Il fallait modifier chaque diagramme en ajoutant l'axe : `ax=axes[i]` et supprimer les `figsize` qui n'étaient plus nécessaires.

La décision de présenter les titres a été également prise (geeksforgeeks, 2024).

Finalement, afin de présenter les deux diagrammes adéquatement et respectant le `figsize`, la fonction `tight_layout` a été utilisée (geeksforgeeks, 2024).

2. De plus, les fonctions **LogisticRegression** ont causé des problèmes, puisqu'elles sont appelées à deux reprises de deux différentes façons (évaluation du modèle et avec la variation croisée). Pour résoudre cette problématique, ces deux appels sont faits juste avant l'exécution du code correspondant.

Bref, afin d'avoir une transparence, toutes les sources web (anciennes et récentes) ont été ajoutées à droite de la règle verticale et une bibliographie a été mise au Google Colab.

Normes et spécifications

GitFlow

L'utilisation de GitFlow a été respectée. Ainsi, la branche «feature/sci1402_thiffaultg» a été créée et utilisée pour l'ensemble des commits.

Commit préfixé

Build

Le commit « ajout du jeu de données UCI » a été préfixé par **build**, puisque celui-ci amène une dépendance externe, soit le jeu de données.

1. [Build: Ajout du jeu de données UCI](#)

Docs

Tous les commits en lien avec l'ajout des travaux pratiques débutent par **docs**, puisqu'ils n'apportent aucun changement au code et qu'ils se veulent être un support à la compréhension du travail.

2. [Docs: Ajout du travail pratique => Préparation de la recherche](#)
3. [Docs: Ajout du travail pratique => Validation du choix de projet](#)
4. [Docs: Ajout du travail pratique => Plan du projet documenté](#)
5. [Docs: Ajout du travail pratique => Rapport mi-parcours](#)
6. [Docs: Ajout du README](#)
7. [Docs: Création de la licence "Creative Commons Legal Code](#)

Feat

Ce commit commence par **feat**, ce qui signifie qu'il ajoute une fonctionnalité, soit le notebook jupyter fait avec Google Colab.

8. [Feat: Ajout du Notenook Jupyter et ses fichiers](#)

À noter que tous les commits ont été rebasés à la fin ce travail. Pour qu'ils aient les bons noms et qu'ils soient dans un ordre logique. Effectivement, après une journée de travail, il est arrivé qu'un nom ne respectant pas les normes ait été donné à un commit.

Pour y parvenir, cette commande a été utilisée : `git rebase -i HEAD~N`. Le N représente le nombre de commits dont l'utilisateur veut modifier. Avec cette option il est possible de les supprimer, renommer, écraser, etc. Important par la suite de pousser les changements avec un `git push --force-with-lease`. Afin de garder un lien avec les commits initiaux.

Fichier ReadMe

Le fichier readme ajouté au répertoire, comprend : l'explication, le but, le contexte du projet sans oublier l'hypothèse.

Structure de dossiers

La structure des dossiers a été respectée et corrigée. En effet, « travail pratique » a été corrigé par « Travaux pratiques », etc. Mais la structure demeure la même.

License

En créant le répertoire sur Github, il était proposé d'ajouter une licence, ce qui n'avait pas été fait au départ. Cependant, ce concept est revenu et le choix de la licence Creative Commons Zero v1.0 Universal a été fait. N'ayant plus l'option pour ajouter une licence, les indications de Github ont été utiles (Documentation Github, 2025)

Pourquoi celle-ci? Pour que le travail effectué durant cette session-ci puisse être partagé au plus grand nombre de personnes et me protège également étant donné qu'aucune garantie n'est émise (Choosealicense, 2025).

Partage du répertoire

Voici le lien du répertoire et de la branche *feature/sci1402_thiffaultg* :

https://github.com/thiffaultg-teluq/sci1402/tree/feature/sci1402_thiffaultg

Une fois l'ensemble des travaux terminés, il y aura un **pull request** vers la branche main.

Bibliographie

Choosealicense. (2025). *Creative Commons Zero v1.0 Universal*. Repéré le 8 juin 2025 à <https://choosealicense.com/licenses/cc0-1.0/>

Geekforgeeks (2024, 23 décembre). Matplotlib.pyplot.tight_layout() in Python. Repéré le 29 mai 2025 à https://www.geeksforgeeks.org/matplotlib-pyplot-tight_layout-in-python/

Geekforgeeks (2024, 5 juillet). Adding Titles to Seaborn Boxplots. Repéré le 29 mai 2025 à <https://www.geeksforgeeks.org/adding-titles-to-seaborn-boxplots/>

Stackoverflow. (2020, 1 février). Draw subplots boxplot using python. Repéré le 29 mai 2025 à <https://stackoverflow.com/questions/60015020/draw-subplots-boxplot-using-python>

Documentation Github. (2025). *Ajout d'une licence à un dépôt*. Repéré le 8 juin 2025 à <https://docs.github.com/fr/communities/setting-up-your-project-for-healthy-contributions/adding-a-license-to-a-repository>

Medium. (2019, 7 octobre). *Git Rebase — how to use interactive rebase properly*. Repéré le 8 juin 2025 à <https://medium.com/fredwong-it/git-rebase-how-to-use-interactive-rebase-properly-34db370be995>